

Day 14 — Mini Build #2

PostgreSQL Analytics Database — Project Guide & Schema Design

Unlike Days 8-13, today has no new theory - it's a build day. The goal is to apply everything from SQL Mastery week (SELECT, joins, CTEs, window functions, optimization) against a schema you design and populate yourself.

1. Schema to Build

Five tables, deliberately connected through foreign keys, modeling a simple e-commerce business:

```
CREATE TABLE customers (  
    customer_id SERIAL PRIMARY KEY,  
    customer_name VARCHAR(100),  
    country VARCHAR(50),  
    signup_date DATE  
);  
  
CREATE TABLE products (  
    product_id SERIAL PRIMARY KEY,  
    product_name VARCHAR(100),  
    category VARCHAR(50),  
    unit_price NUMERIC(10,2)  
);  
  
CREATE TABLE orders (  
    order_id SERIAL PRIMARY KEY,  
    customer_id INT REFERENCES customers(customer_id),  
    order_date DATE,  
    status VARCHAR(20)  
);  
  
CREATE TABLE order_items (  
    order_item_id SERIAL PRIMARY KEY,  
    order_id INT REFERENCES orders(order_id),  
    product_id INT REFERENCES products(product_id),  
    quantity INT,  
    unit_price NUMERIC(10,2)  
);  
  
CREATE TABLE payments (  
    payment_id SERIAL PRIMARY KEY,  
    order_id INT REFERENCES orders(order_id),  
    payment_date DATE,  
    amount NUMERIC(10,2),  
    payment_method VARCHAR(30)  
);
```

Why split into 5 tables instead of one flat table?

- **orders** vs. **order_items**: one order can contain many products - this is a one-to-many relationship that a flat table cannot represent cleanly
- **payments** separate from **orders**: an order could theoretically be paid in more than one transaction, or refunded - keeping it separate models reality more accurately
- This is the same normalization principle from Day 21-22, now applied to a schema you build yourself instead of just reading about

Key takeaway: Don't index every foreign key by default, but for this exercise, add indexes on orders.customer_id, order_items.order_id, and payments.order_id - these are the columns you'll join on constantly.

2. Populating the Tables

Use enough sample data that your queries return meaningful results - a handful of rows won't reveal much. A reasonable target:

- 20-30 customers across a few different countries
- 15-20 products across 3-4 categories
- 100+ orders spread across several months (so month-over-month queries have something to show)
- Multiple order_items per order (so joins actually multiply rows meaningfully)
- Some orders with no payment yet, and some fully paid - to make status-based queries realistic

Tip: generate this with a Python script (Day 1-6 skills) instead of typing INSERT statements by hand - loop and generate randomized but plausible data, then load it with psycopg2 (Day 20) or export to CSV and COPY it in.

3. The 25-30 Required Queries — By Category

a) Joins (5+ queries)

```
-- Every order with customer name and product details
SELECT o.order_id, c.customer_name, p.product_name, oi.quantity
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id;
```

b) CTEs (5+ queries)

```
-- Total spend per customer, using a CTE, filtered to high spenders
WITH customer_spend AS (
    SELECT o.customer_id, SUM(oi.quantity * oi.unit_price) AS total_spent
    FROM orders o
    JOIN order_items oi ON o.order_id = oi.order_id
    GROUP BY o.customer_id
)
SELECT c.customer_name, cs.total_spent
FROM customer_spend cs
JOIN customers c ON c.customer_id = cs.customer_id
WHERE cs.total_spent > 5000;
```

c) Window Functions / Ranking (5+ queries)

```
-- Rank products by revenue within each category
SELECT
    p.category,
    p.product_name,
    SUM(oi.quantity * oi.unit_price) AS revenue,
    RANK() OVER (
        PARTITION BY p.category
        ORDER BY SUM(oi.quantity * oi.unit_price) DESC
    ) AS category_rank
FROM products p
JOIN order_items oi ON p.product_id = oi.product_id
GROUP BY p.category, p.product_name;
```

d) Revenue Queries (5+ queries)

```
-- Monthly revenue trend
SELECT
    DATE_TRUNC('month', o.order_date) AS month,
    SUM(oi.quantity * oi.unit_price) AS monthly_revenue
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
GROUP BY DATE_TRUNC('month', o.order_date)
ORDER BY month;
```

e) Retention / Customer Behavior (5+ queries)

```
-- Customers who ordered in both January and February
SELECT customer_id
FROM orders
WHERE DATE_TRUNC('month', order_date) = '2026-01-01'
INTERSECT
SELECT customer_id
FROM orders
WHERE DATE_TRUNC('month', order_date) = '2026-02-01';
```

Key takeaway: Organize your final SQL file by category (joins, CTEs, window functions, revenue, retention) with a comment above each query explaining the business question it answers - this file becomes something you can talk through in an interview.

Day 14 Summary — Mini Build #2 Checklist

Deliverable	Status Check
5 tables created	customers, products, orders, order_items, payments
Foreign keys defined	orders->customers, order_items->orders/products, payments->orders
Indexes added	On the foreign key columns used in joins
Realistic sample data	20-30 customers, 15-20 products, 100+ orders across months
25-30 queries written	Covering joins, CTEs, window functions, revenue, retention
Queries organized & commented	Grouped by category, each explaining its business question

Overall takeaway: This project is the proof that Days 8-13 actually stuck - not by re-reading the syntax, but by designing a schema from scratch and writing real business-question queries against it, the same workflow used in an actual analytics or data engineering role.